SIMATS ENGINEERING

ASSESSMENT TOOL 2

Name: Thirupugazh S R
Reg No: 192371016
Marks: 92/100

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

Name: Thirupugazh S R
Reg No: 192371016

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I ***Thirupugazh S R / 192371016*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Thirupugazh S R**

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Estimated Score
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	23/25
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	23/25
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	18/20
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	18/20
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	10/10
Total	**100%**	**Estimated Total**				**92/100**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

Answer 1 – Incorrect or invalid fields

Header line	Valid?	Interpretation
HTTP/1.1 200 OK	Valid	Correct status-line syntax: protocol version, status code 200, reason phrase.
Date: Mon, 01 Apr 2026 10:00:00 GMT	Invalid	The format is correct, but the day name does not match the date: 1 April 2026 is a Wednesday, not a Monday.
Content-Type: text/html	Valid (incomplete)	Syntactically valid, but it does not state the character set; text/html; charset=UTF-8 is recommended.
Content-Length: -150	Invalid	The length of a message body is a count of bytes, so it must be a non-negative integer. A negative number is meaningless.
Connection: keep-alive	Valid	Asks to keep the TCP connection open (the default in HTTP/1.1). It makes an accurate Content-Length more important.

Answer 2 – Issue with Content-Length and its impact

Content-Length tells the client exactly how many bytes of body to read. The value -150 cannot be a byte count, so the **message framing is broken**: the client cannot tell where this response ends.

- With **keep-alive**, the same connection is reused for the next response, so a wrong length can make the client wait for bytes that never arrive (page hangs) or start reading the next response in the middle of this body.
- Under the HTTP/1.1 rules (RFC 9112), a response with an invalid Content-Length is an unrecoverable framing error: a user agent should discard it and close the connection. In our test, **curl** rejected it with "curl: (8) Invalid Content-Length: value" (see Output Screen).
- Some browsers are tolerant: Chromium ignored the bad value and showed the page. Behaviour therefore differs between clients, which makes the fault unpredictable.
- Mismatched lengths between proxies and servers can also lead to request-smuggling / cache-poisoning security problems, and the user may see truncated or blank pages.

Answer 3 – Does the status code match the response correctness?

The status line "200 OK" is valid by itself, and 200 is the right code for a successful retrieval. However, a 200 promises that the representation was delivered correctly. Because the headers are

malformed and the body cannot be framed reliably, the response as a whole is **not correct**, so the status code does not match the real condition of the response. The server should have produced a correct 200 with a valid length, or (if it failed internally) a 5xx code such as 500 Internal Server Error.

Answer 4 – Corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 14
Connection: keep-alive
```

- **Date:** use the correct day name (Wed), or generate the value automatically from the server clock in IMF-fixdate format.
- **Content-Length:** must equal the exact number of body bytes (14 for <h1>Hello</h1>); better, let the web server compute it, or use Transfer-Encoding: chunked for streamed content.
- **Content-Type:** add charset=UTF-8 so the text is decoded correctly.

Conclusion

The header contains two definite errors (negative Content-Length and a wrong weekday in Date) and one weakness (no charset). The negative Content-Length is the serious one because it breaks message framing and can make clients fail or behave differently, even though the status code says 200 OK.

Output Screen – Question 1

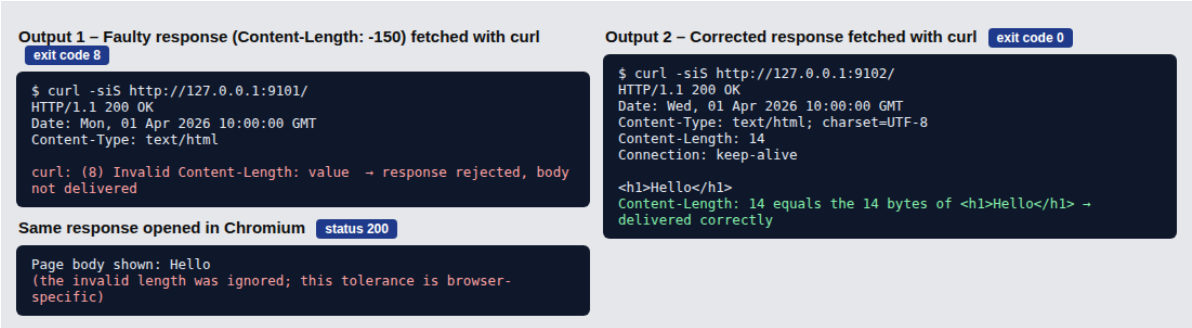


Figure 1: Faulty response rejected by curl (exit code 8) and tolerated by Chromium; corrected response delivered correctly.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

Answer 1 – Missing or improperly closed elements

- ** HelloWorld** has no closing tag. HTML5 allows the browser to infer it, but it is not allowed in XHTML/XML and is poor practice.
- The arrives while is still open, so the elements are not closed in reverse order of opening (improper nesting).
- The page has no **<!DOCTYPE html>**, no lang attribute on <html>, and no <meta charset>.
- There are no semantic elements (header, main, footer); everything is placed in a generic .

Answer 2 – How the browser interprets the structure

The browser does not reject the page. Its HTML parser follows error-recovery rules: when it meets while a is open, it first closes the automatically (implied end tag) and then closes the . The DOM it builds (captured from Chromium) is:

```
<html><head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</p></div>
</body></html>
```

Because there is **no DOCTYPE**, the browser switches to **quirks mode** (document.compatMode = "BackCompat"), whereas the corrected page runs in standards mode ("CSS1Compat"). An XML/XHTML parser would not repair the page at all: it would stop with a well-formedness error.

Answer 3 – Impact on rendering and layout

- For this tiny page the visible text looks the same in both versions (see Output Screen), because the parser repairs the missing .
- In quirks mode the box model, line-height, default font sizes in tables and some margins are calculated differently, so real pages with this fault can look different from browser to browser.
- Missing lang and charset can cause wrong hyphenation, wrong screen-reader pronunciation and garbled characters.

- Relying on the parser to repair errors makes the page harder to maintain and style with CSS and JavaScript, because the DOM may not be the one the author expects.

Answer 4 – Corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>HelloWorld</p>
  </div>
</body>
</html>
```

Conclusion

The DOM is repaired by the browser, but the markup is not well-formed: the paragraph is not closed and the page runs in quirks mode. Adding </p>, the DOCTYPE, lang and charset gives a valid, predictable DOM in every browser.

Output Screen – Question 2

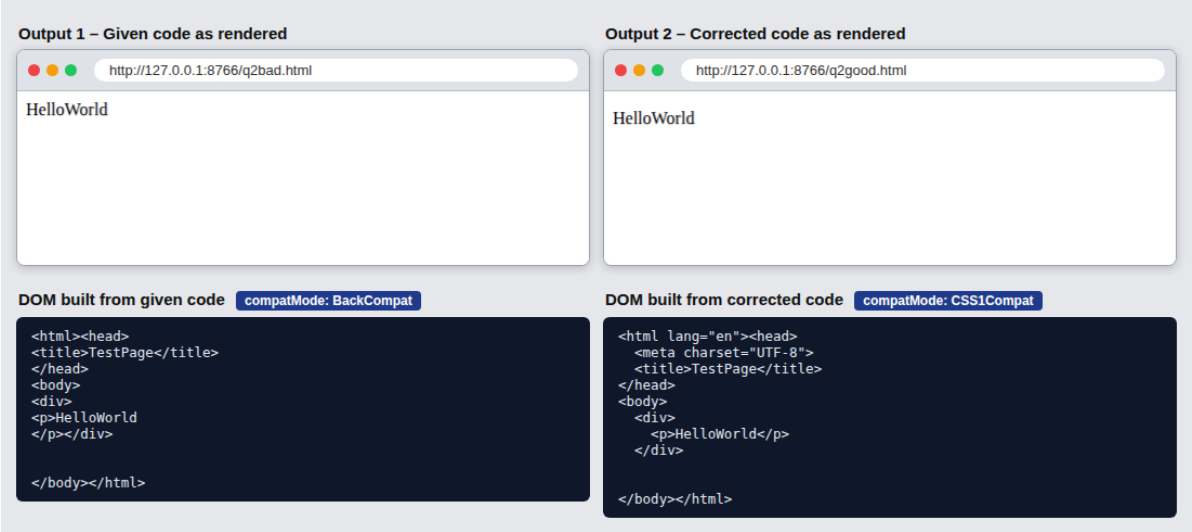


Figure 2: Given vs corrected code as rendered, with the DOM and rendering mode built by the browser.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

Answer 1 – Differences in data transmission

Aspect	GET	POST
Where data travels	In the URL query string (? user=admin&pass=1234)	In the request body (Content-Type: application/x-www-form-urlencoded)
Visible in address bar / history	Yes	No
Purpose	Retrieve data; safe and idempotent	Send data that changes server state
Caching / bookmarks	Can be cached and bookmarked	Normally not cached or bookmarked
Size limit	Limited by URL length	Practically unlimited
Server log entry	Contains full URL with data	Contains only /login (body not logged by default)

In the demonstration (Output Screen), the GET login put user=admin&pass=1234 in the address bar and in the server access log, while the POST login showed only /login and carried the data in the body.

Answer 2 – Which method is more suitable for login, and why

POST is more suitable. A login sends sensitive credentials and causes the server to create a session (a state change), which matches the purpose of POST. POST keeps the credentials out of the URL, browser history, bookmarks, cache and normal access logs. It is only "Moderate", not "High", because the body is still readable by anyone on the network unless the connection is encrypted with HTTPS.

Answer 3 – Security risks of using GET for authentication

- The password is displayed in the address bar and can be seen by anyone looking at the screen (shoulder surfing).
- URLs are saved in browser history, bookmarks and auto-complete suggestions.
- URLs are written to web-server, proxy and firewall logs, so passwords are stored in plain text there.
- The URL is passed in the Referer header to other sites and can be leaked through shared links.
- Cached or prefetched GET requests can be repeated or replayed by others, and URLs can be sent to others by mistake.

Answer 4 – Best practices for a secure login implementation

- Use **POST over HTTPS (TLS)** only, and redirect all HTTP traffic to HTTPS with HSTS.

- Store passwords as salted hashes (bcrypt, scrypt or Argon2); never store or log plain-text passwords.
- Protect the form with a **CSRF token**; set session cookies with HttpOnly, Secure and SameSite flags.
- Validate and sanitise input on the server (prevents SQL injection and XSS).
- Limit attempts (rate limiting / lock-out / CAPTCHA) and consider multi-factor authentication.
- Show generic error messages ("Invalid username or password") and regenerate the session ID after login.
- Use autocomplete="username" / "current-password" and type="password" so password managers work and the characters are masked.

Conclusion

GET exposes credentials in many places, so it must never be used for authentication. POST is the correct method, and together with HTTPS and the practices above it makes the login process secure.

Output Screen – Question 3

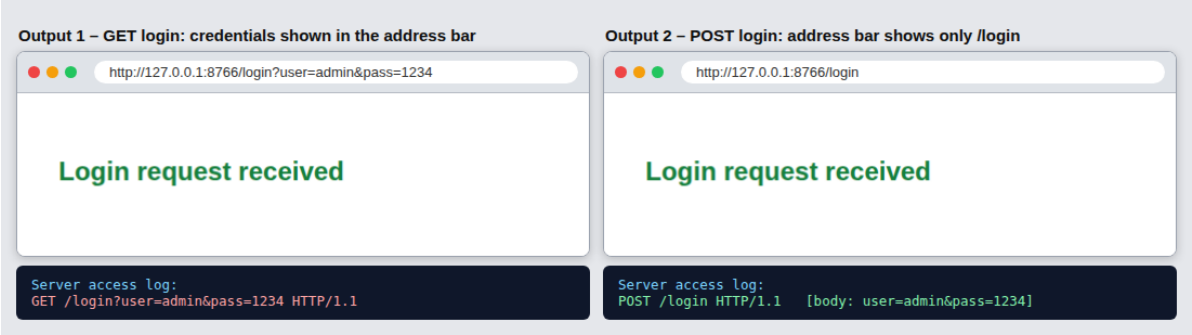


Figure 3: GET login shows the credentials in the URL and log; POST login does not.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

Answer 1 – Missing closing tags

- **<h1>Welcome** is never closed: </h1> is missing.
- ** This is a paragraph** is never closed: is missing (allowed to be omitted in HTML5, but required in XHTML).
- <img> is a void element and needs no closing tag in HTML5; in XHTML it must be written as .

Answer 2 – Issues with the <img> tag

- The **alt** attribute is missing. It is required by HTML5 validation; it supplies text for screen readers and is shown when the image cannot load (accessibility and SEO).
- No **width and height** are given, so the browser cannot reserve space and the layout may jump when the image loads.
- The image is placed inside the unclosed (and <h1>), so it becomes part of the heading text flow.
- The file image.jpg does not exist in the test, so a broken-image icon appears (see Output Screen); a missing file should be handled with proper alt text.
- XHTML would need the self-closing form .

Answer 3 – How browsers handle these errors

Browsers do not stop on these errors; they apply HTML5 parsing rules. A start tag does not close an open <h1>, so the paragraph and the image are placed **inside** the heading until </body> ends everything. The DOM captured from Chromium was:

```
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">



</p></h1>
```

The result is that "This is a paragraph" is displayed in large bold heading style. Older or non-standard engines may repair the markup differently, and an XML parser would give an error. The missing image shows a broken-image icon because there is no alt text.

Answer 4 – Corrected version of the code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Sample image" width="200" height="120">
</body>
</html>
```

Conclusion

The snippet is rendered, but only because browsers repair it: the heading swallows the paragraph and the image. Closing <h1> and <p>, and giving <img> alt, width and height, gives valid, accessible and predictable output.

Output Screen – Question 4

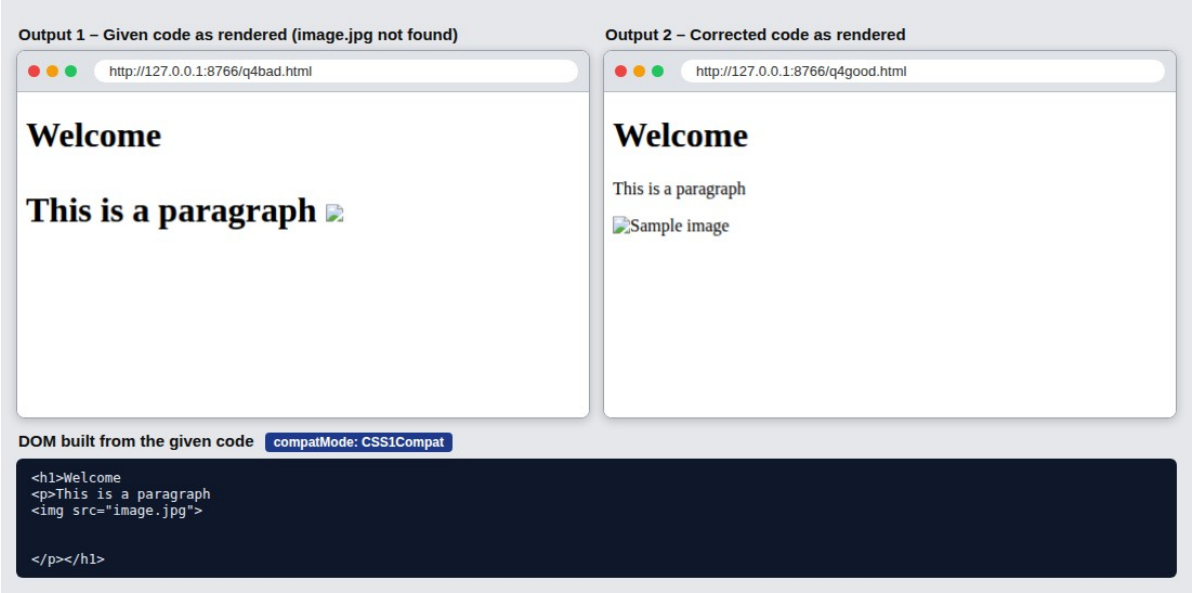


Figure 4: Given code (paragraph shown as heading) vs corrected code, with the DOM built by the browser.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Answer 1 – Interpretation of each step

Step	What happens
1. User enters a URL	The browser splits the URL into scheme (https), host (www.example.com), path and query. It checks its cache, HSTS list and history before going to the network.
2. DNS lookup	The host name is converted to an IP address using browser, OS, router, and then the ISP/recursive resolver (root, TLD and authoritative servers if not cached).
3. HTTP request is sent	The browser opens a TCP connection (and a TLS handshake for HTTPS) and sends the request line, headers (Host, User-Agent, Accept, cookies) and body if any.
4. Server processes request	The web server routes the URL, runs application code, queries the database or files and prepares the response.
5. Response is returned	The server sends the status line (e.g. 200 OK), headers (Content-Type, Content-Length, Cache-Control) and the body (HTML).
6. Browser renders page	The browser parses HTML into the DOM, CSS into the CSSOM, requests images/CSS/JS, runs scripts, builds the render tree, lays out and paints the page.

Answer 2 – Where delays can occur

- **DNS lookup:** uncached lookups need several network round trips.
- **TCP/TLS connection set-up:** each handshake costs one or more round-trip times, especially over long distances.
- **Network latency and bandwidth:** distance to the server, slow mobile links and large responses.
- **Server processing (time to first byte):** slow code, unindexed database queries, overloaded server.
- **Download of the page and its resources:** many requests, large images, uncompressed files, redirects.
- **Rendering:** large DOM, render-blocking CSS/JS and heavy scripts delay the first paint.

Answer 3 – Role of DNS

DNS (Domain Name System) works like the Internet's phone book. Browsers and servers communicate using IP addresses, but people use names, so DNS translates www.example.com into an address such as 93.184.216.34 before any connection can be made. It is step 2 and **no HTTP request can be sent until it finishes**. Results are cached (controlled by the record TTL) in the browser, operating system and resolver, which is why repeat visits are faster. DNS also provides load balancing/geo-routing (different IPs for different users) and failover, and a slow or failing DNS server makes a site appear to be down even if the web server is working.

Answer 4 – Optimizations to improve performance

- Reduce DNS time: low-TTL-aware caching, a fast DNS provider, <link rel="dns-prefetch"> and <link rel="preconnect"> for important hosts, and fewer third-party domains.
- Use **HTTP/2 or HTTP/3**, keep-alive connections and TLS 1.3 to cut connection set-up.
- Use a **CDN** to serve content from a location near the user.
- Enable **caching** (Cache-Control, ETag) in the browser and server; use server-side caching and database indexes to lower processing time.
- Compress text with **gzip/Brotli**, minify HTML/CSS/JS and optimise and lazy-load images (WebP/AVIF).
- Avoid redirects, load non-critical JavaScript with defer/async and inline critical CSS to speed up rendering.

Conclusion

A page load is a chain of six dependent steps, so a delay in any one of them delays the whole page. DNS and connection set-up add network delay, server processing adds waiting time, and rendering adds client-side delay. Optimising each stage with caching, CDN, compression and modern protocols gives the biggest improvement.

Output Screen – Question 5

Output 1 – Browser–server communication flow

1 URL entered → 2 DNS lookup → 3 TCP + HTTP request → 4 Server processing → 5 Response returned → 6 Browser renders

Output 2 – Measured timings for one page load (PerformanceNavigationTiming, local test server)

Phase	Step	Measured (ms)
DNS lookup	2	0.0
TCP connect	3	0.4
Request sent → first byte (server processing)	3–4	0.9
Response download	5	3.5
Parse HTML / DOM ready	6	23.8
Total until load event	1–6	**43.8**

Note: the test server is on localhost, so DNS and connect times are about 0 ms. On the real Internet these phases are usually where most delay appears.

Figure 5: The six-step flow, and measured timings of one real page load on a local test server.